Source

Source - Daniel Roesler, GitHub.

- Published: date not stated
- Original: <https://github.com/diafygi/webrtc-ips>
- Preserved from: https://github.com/diafygi/webrtc-ips (git) on 2026-08-08
- Repository commit: ba63ef512c5f4bb0ac798679bef5cab9b71efc4f
- Licence: see the repository

Content

> UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

This reference is a source-code repository. The archive preserves its documentation at an exact commit; the code itself stays in a private mirror and is never checked out, built or run.

- Repository: <https://github.com/diafygi/webrtc-ips>
- Commit: `ba63ef512c5f4bb0ac798679bef5cab9b71efc4f`
- Documents preserved: 2

LICENSE

Blob `99dc03805814`, 1082 bytes, at commit `ba63ef512c5f`.

ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

README.md

Blob `f2517fac5635` *, 3393 bytes, at commit* `ba63ef512c5f` *.*

STUN IP Address requests for WebRTC

Demo: https://diafygi.github.io/webrtc-ips/

What this does

Firefox and Chrome have implemented WebRTC that allow requests to STUN servers be made that will return the local and public IP addresses for the user. These request results are available to javascript, so you can now obtain a users local and public IP addresses in javascript. This demo is an example implementation of that.

Additionally, these STUN requests are made outside of the normal XMLHttpRequest procedure, so they are not visible in the developer console or able to be blocked by plugins such as AdBlockPlus or Ghostery. This makes these types of requests available for online tracking if an advertiser sets up a STUN server with a wildcard domain.

Code

Here is the annotated demo function that makes the STUN request. You can copy and paste this into the Firefox or Chrome developer console to run the test.

```
//get the IP addresses associated with an account
function getIPs(callback){
    var ip_dups = {};

    //compatibility for firefox and chrome
    var RTCPeerConnection = window.RTCPeerConnection
        || window.mozRTCPeerConnection
        || window.webkitRTCPeerConnection;
    var useWebKit = !!window.webkitRTCPeerConnection;

    //bypass naive webrtc blocking using an iframe
    if(!RTCPeerConnection){
        //NOTE: you need to have an iframe in the page right above the script tag
        //
        //<iframe id="iframe" sandbox="allow-same-origin" style="display: none"></iframe>
        //<script>...getIPs called in here...
        //
        var win = iframe.contentWindow;
        RTCPeerConnection = win.RTCPeerConnection
            || win.mozRTCPeerConnection
            || win.webkitRTCPeerConnection;
        useWebKit = !!win.webkitRTCPeerConnection;
    }

    //minimal requirements for data connection
    var mediaConstraints = {
        optional: [{RtpDataChannels: true}]
    };

    var servers = {iceServers: [{urls: "stun:stun.services.mozilla.com"}]};

    //construct a new RTCPeerConnection
    var pc = new RTCPeerConnection(servers, mediaConstraints);

    function handleCandidate(candidate){
        //match just the IP address
        var ip_regex = /([0-9]{1,3}(\.[0-9]{1,3}){3}|[a-f0-9]{1,4}(:[a-f0-9]{1,4}){7})/
        var ip_addr = ip_regex.exec(candidate)[1];

        //remove duplicates
        if(ip_dups[ip_addr] === undefined)
            callback(ip_addr);

        ip_dups[ip_addr] = true;
```

```
    }

    //listen for candidate events
    pc.onicecandidate = function(ice){

      //skip non-candidate events
      if(ice.candidate)
        handleCandidate(ice.candidate.candidate);
    };

    //create a bogus data channel
    pc.createDataChannel("");

    //create an offer sdp
    pc.createOffer(function(result){

      //trigger the stun server request
      pc.setLocalDescription(result, function(){}, function(){});

    }, function(){});

    //wait for a while to let everything done
    setTimeout(function(){
      //read candidate info from local description
      var lines = pc.localDescription.sdp.split('\n');

      lines.forEach(function(line){
        if(line.indexOf('a=candidate:') === 0)
          handleCandidate(line);
      });
    }, 1000);
}

//Test: Print the IP addresses into the console
getIPs(function(ip){console.log(ip);});
```